ME2605 / DDA6204 · Neural Net Training · Fall 2026

Lecture 5: Normalization and the Convergence Rate

Ruoyu Sun

OPENING

Today's Plan

- Section 1: other normalization layers.
- Section 2: convergence rate of gradient descent.
- Section 3: analyzing a regression example.
- Section 4: understanding normalization layers using condition number.

OPENING

Learning Goals

- Explain what weight normalization and spectral normalization are.
- Apply the rate of gradient descent, and check the conditions it needs.
- Describe L-smoothness, μ-strong convexity and the condition number κ.
- Describe data preprocessing and explain why it works.
- Derive the normalization layer from the perspective of reducing κ.

PREVIOUSLY

Review of Lecture 4

- **Motivation (Lec 3) —** large depths: training wants the signal to propagate well.
- **NO is a layer —** inserted like an activation: differentiable, consistent with backprop.
- **NO definition (Lec 4) —** standardize by computed μ, σ; then learnable γ, β.
- **Two variants —** NO-S: scalar γ, β.
  NO-V: featurewise γ, $\beta \in \mathbb{R}^d$.
- **LayerNorm —** NO-V on each sample — each column.
- **Gradients —** the LayerNorm gradient is computed by backpropagation; the formula adds one extra matrix — identity minus two rank-one matrices.
- **BatchNorm —** NO-S on each feature row — each row, across the batch.
- **Training and inference —** for BatchNorm the two modes may have a mismatch; there are a few methods to compute the statistics.

OPENING

One More Week on Normalization

Why one more week on normalization?

Technical reason 1 — we can compute it, but not explain it

- we know how to compute every normalization layer; we have not said why it helps.

Technical reason 2 — other normalization layers

- we only introduced LayerNorm, RMSNorm and BatchNorm.
- WN and SN have not been introduced yet.

High level — why it matters

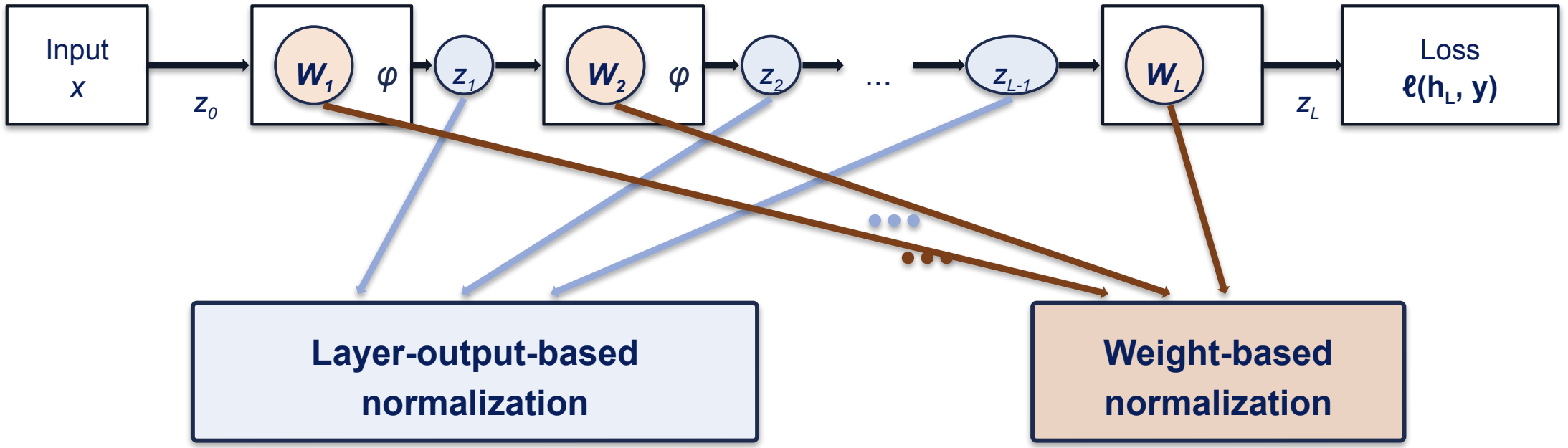
- normalization is one of the most important components in a modern network.
- LM companies keep tuning the normalization tricks (DeepSeek, Kimi, etc.).

SECTION 1

Other Normalization Layers

Two Types of Normalization Layers

Normalization can act on activations or on the weights that produce them.

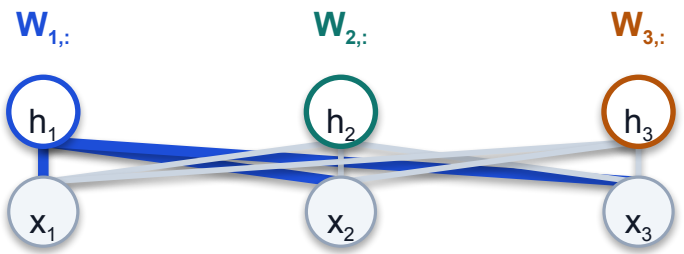


Layer normalization (transformers)
Batch normalization (image classification)
Group normalization
…

Weight normalization
Spectral normalization (popular in GAN)

WeightNorm: Normalize Each Row of W

Next, we briefly introduce weight normalization.
WeightNorm splits each row of W into a direction and a scale.



$$W_{j,:} = g_j \frac{V_{j,:}}{\|V_{j,:}\|_2}, \qquad V_{j,:} \neq 0$$

g_j is a learnable parameter, one for each row of W.

The norm of the j-th row of W is g_j.

Reminder: Singular Values

Before the definition of spectral norm, we review singular values.

$$A \in \mathbb{R}^{m \times n} \Rightarrow \text{ singular values } \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k, \quad k = \min(m, n)$$

$$\sigma_i = \sqrt{\lambda_i(A^\top A)}, \quad i = 1, \ldots, k$$

$$A^\top A \ (n \times n) \text{ has eigenvalues } \sigma_1^2, \ \sigma_2^2, \ \ldots, \ \sigma_k^2, \ 0, \ \ldots, \ 0$$

The singular values are the square roots of the eigenvalues of A^TA.

A^TA is n × n; if it has more eigenvalues than min(m, n), keep only the first k — the rest are zero.

SpectralNorm: Normalize the Whole Matrix

SpectralNorm is an operation on the matrix W.

This operation is embedded in the neural network.

The definition: divide the whole matrix by its largest singular value.

$$\sigma_{\max}(W) = \sigma_1(W) \qquad \text{the largest singular value of } W$$

$$\overline{W} = \frac{W}{\sigma_{\max}(W)}, \qquad W \neq 0$$

SpectralNorm normalizes the weight.

It does not center it.

SpectralNorm: Two Properties

There are two properties of the newly defined matrix $\bar{W}$.

$$\sigma_1(\overline{W}) = 1$$

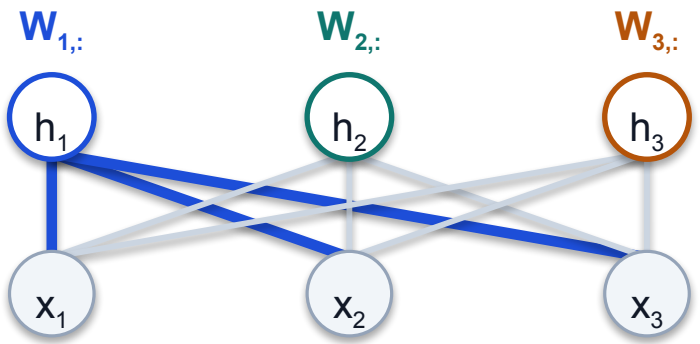
$$\|\overline{W}u\|_2 \leq \|u\|_2 \qquad \text{for every vector } u$$

- Question: how is σ_{max} computed in practice?
- Answer: power iteration converges to the largest singular value — in practice one step, or a few (say 10), of power iteration.

Rows or Columns?

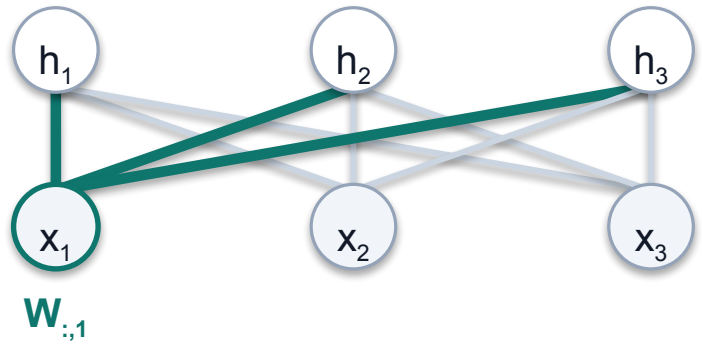
A row of W is the incoming weights of one output neuron.
A column of W is the outgoing weights of one input neuron.

Row: one output neuron's weights



WN (2016), CWN (2017), WS (2019)

Column: one input neuron's weights

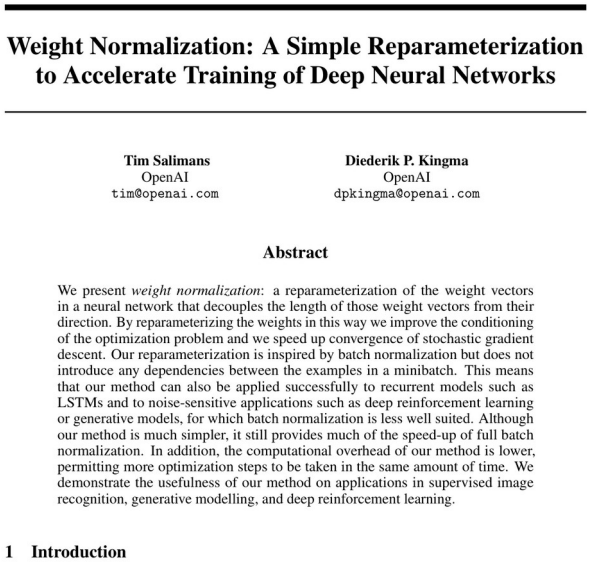


I am not aware of a famous paper on this

THE SOURCES

The Original Papers

We show the two papers below.



Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks

Tim Salimans
OpenAI
tim@openai.com

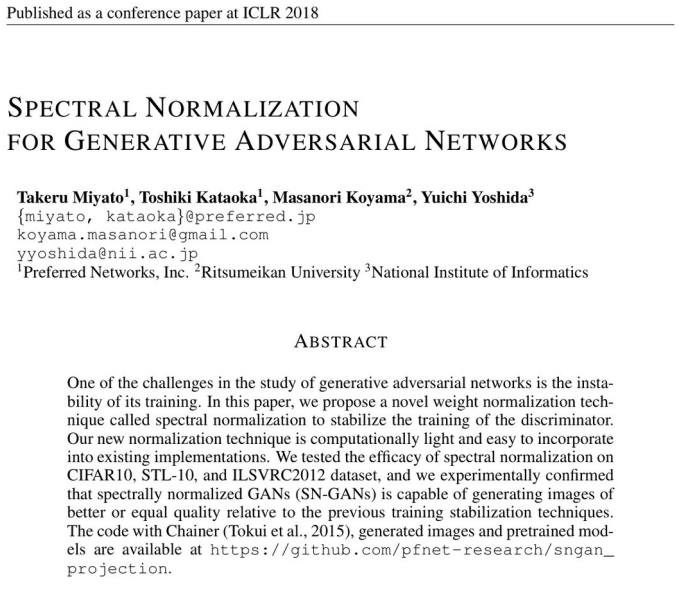
Diederik P. Kingma
OpenAI
dpkingma@openai.com

Abstract

We present *weight normalization*: a reparameterization of the weight vectors in a neural network that decouples the length of those weight vectors from their direction. By reparameterizing the weights in this way we improve the conditioning of the optimization problem and we speed up convergence of stochastic gradient descent. Our reparameterization is inspired by batch normalization but does not introduce any dependencies between the examples in a minibatch. This means that our method can also be applied successfully to recurrent models such as LSTMs and to noise-sensitive applications such as deep reinforcement learning or generative models, for which batch normalization is less well suited. Although our method is much simpler, it still provides much of the speed-up of full batch normalization. In addition, the computational overhead of our method is lower, permitting more optimization steps to be taken in the same amount of time. We demonstrate the usefulness of our method on applications in supervised image recognition, generative modelling, and deep reinforcement learning.

1 Introduction

Salimans & Kingma, NeurIPS 2016 - cited by 2,855



Published as a conference paper at ICLR 2018

SPECTRAL NORMALIZATION FOR GENERATIVE ADVERSARIAL NETWORKS

Takeru Miyato[1], Toshiki Kataoka[1], Masanori Koyama[2], Yuichi Yoshida[3]
{miyato, kataoka}@preferred.jp
koyama.masanori@gmail.com
yyoshida@nii.ac.jp
[1]Preferred Networks, Inc. [2]Ritsumeikan University [3]National Institute of Informatics

ABSTRACT

One of the challenges in the study of generative adversarial networks is the instability of its training. In this paper, we propose a novel weight normalization technique called spectral normalization to stabilize the training of the discriminator. Our new normalization technique is computationally light and easy to incorporate into existing implementations. We tested the efficacy of spectral normalization on CIFAR10, STL-10, and ILSVRC2012 dataset, and we experimentally confirmed that spectrally normalized GANs (SN-GANs) is capable of generating images of better or equal quality relative to the previous training stabilization techniques. The code with Chainer (Tokui et al., 2015), generated images and pretrained models are available at https://github.com/pfnet-research/sngan_projection.

Miyato et al., ICLR 2018 - cited by 7,196

Citation counts: Google Scholar, October 2026.

BatchNorm (Ioffe & Szegedy, 2015) stands at 69,710 for scale.

COMPARISON

Comparison of Four Normalization Layers

Comparison of four common normalization layers.
Same operation, two axes — plus the two weight-based members, side by side:

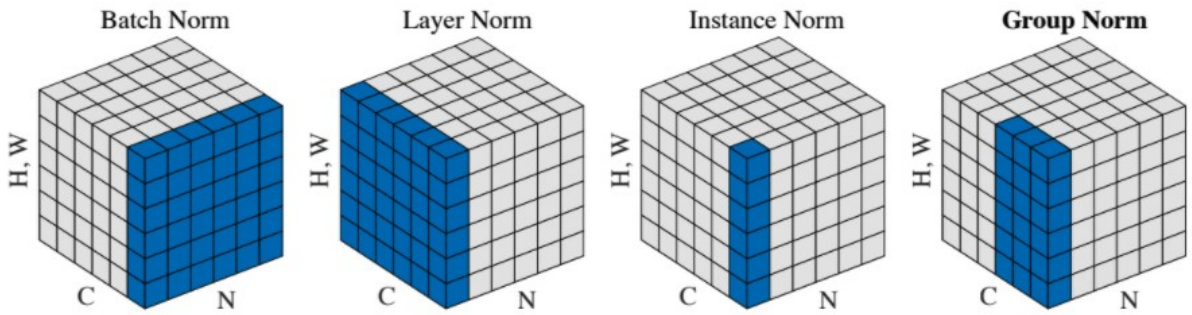
	BatchNorm	LayerNorm	WeightNorm	SpectralNorm
Paper	Ioffe & Szegedy, 2015 (cited by 69,710)	Ba, Kiros & Hinton, 2016 (cited by 20,646)	Salimans & Kingma, 2016 (cited by 2,855)	Miyato et al., 2018 (cited by 7,196)
Normalizes	activations — per feature, across the batch	activations — per sample, across the features	weights — one row (neuron) at a time	weights — the whole matrix, divided by σ_{max}
Train vs test	two functions (running statistics)	identical	identical	identical
Needs a batch	yes — a fair batch	no — any size, even 1	no	no
Where used	CNN vision, large batches	RNN, Transformers	few papers advocate it — the same idea reappears inside other methods	GAN discriminators

the four members in one table

ON TENSORS

Other Normalization Layers for Tensors

So far the network has been an MLP, and the normalization layers act on a matrix.
In a convolutional network, a conv layer's activations form a tensor — a 4-dim array in Python.
Different ways of grouping the entries give different normalization layers.
One operation, four ways:



which vectors are standardized, for each layer type

Blue marks the values that share one mean and variance.
N is the sample count; C channels; H and W the spatial dimensions.

Changing the group changes which samples or features influence each other.

SECTION 2

Convergence of Gradient Descent

Section 2 in Four Parts

2.1 Convergence

- when gradient descent converges at all: the setting, the algorithm, and the stepsizes the theory allows.

2.2 Convergence Speed

- the rate, and the iteration count that follows from it.

2.3 Why the Rate Has This Form

- the two mechanisms -- where the step ceiling comes from, and where the condition number comes from -- and the zig-zag picture.

2.4 Two Paradoxes

- where the bound and the measured runs disagree, and what survives without strong convexity.

SECTION 2.1

Convergence

the setting, the algorithm, and the stepsizes the theory allows

Why We Need a Convergence Rate

To understand normalization layers deeply, we need some convergence theory.

- A convergence rate relates the number of updates to a clearly defined error.

- This analysis explains one mechanism; it does not account for every benefit of normalization.

SETTING

What We Minimize

Let F(w) be a twice continuously differentiable training objective, minimized over the parameter vector w:

Problem: $\min_{w \in \mathbb{R}^P} F(w), \qquad F : \mathbb{R}^P \to \mathbb{R}$ twice differentiable.

- Remark: GeLU, SwiGLU and the activations used in practice are smooth — twice differentiability holds for real networks.
- ReLU is the exception: a non-smooth point at zero.
  A small gap here.
- But ReLU is not much used any more, so we set it aside and do not discuss it.

The Two Kinds of Minimum

Local and global minimum

$$\textbf{Local minimum } \hat{w}: \quad \exists \varepsilon > 0 \text{ such that } F(w) \geq F(\hat{w}) \text{ for all } \|w - \hat{w}\| \leq \varepsilon$$

$$\textbf{Global minimum } w^\star: \quad F(w) \geq F(w^\star) \text{ for all } w \in \mathbb{R}^P$$

A local minimum has no smaller value in some neighbourhood; a global minimum, no smaller value anywhere.

Necessary condition for local minimum

$$\nabla F(w) = 0 \qquad \text{and} \qquad \nabla^2 F(w) \succeq 0$$

the gradient vanishes and the Hessian is positive semidefinite

Sufficient condition for local minimum

$$\nabla F(w) = 0 \qquad \text{and} \qquad \nabla^2 F(w) \succ 0$$

the gradient vanishes and the Hessian is positive definite

- Strict versions replace ≥ by >.

Convexity

Convexity

$$F(\theta w + (1-\theta)v) \leq \theta F(w) + (1-\theta)F(v) \qquad \text{for all } w, v, \ 0 \leq \theta \leq 1$$

For twice differentiable functions this is equivalent to the Hessian being positive semidefinite everywhere.

Sufficient condition for global minimum

$$\text{If } F \text{ is convex, } \ \nabla F(w) = 0 \implies w \text{ is a global minimum of } F$$

Every stationary point of a convex function is a global minimum.



one stationary point — the global minimum



a stationary point that is not the global minimum

A deep network is not convex, but the convex case can be a starting point for us to understand.

L-smooth and μ-Strongly-Convex

Two constants bound the curvature from above and from below:

L-smooth

$$\|\nabla F(w) - \nabla F(v)\| \leq L\|w - v\| \qquad \text{for any } w, v$$

$$\implies \lambda_{\max}\left(\nabla^2 F(w)\right) \leq L \qquad \text{for every } w$$

L bounds how quickly the gradient changes.

μ-strongly convex

$$\langle \nabla F(w) - \nabla F(v), w - v \rangle \geq \mu\|w - v\|^2 \qquad \text{for any } w, v$$

$$\implies \lambda_{\min}\left(\nabla^2 F(w)\right) \geq \mu \qquad \text{for every } w$$

μ > 0 gives a curvature lower bound.

- Example: for a positive definite quadratic $F(w) = \frac{1}{2}w^T Q w$, Q is the Hessian, so $L = \lambda_{max}(Q)$ and $\mu = \lambda_{min}(Q)$.

Review: Gradient Descent

The algorithm, formally:

$$w[0] \text{ given}, \qquad \alpha > 0, \qquad r = 0, 1, 2, \ldots$$

$$w[r+1] = w[r] - \alpha \nabla F(w[r])$$

One step costs one gradient evaluation.

- No Hessian is formed or inverted; the stepsize α can be tuned.

A natural question: how many steps does gradient descent need to achieve a certain error?

Stepsize: What the Theory Allows

Proposition 1. For an L-smooth F, using GD with either one of the following stepsizes:

(1) $\varepsilon < \alpha[r] \leq \dfrac{2-\varepsilon}{L}$ for some $\varepsilon \in (0,2)$, for all r

(2) $\alpha[r] \to 0$ and $\displaystyle\sum_{r=1}^{\infty} \alpha[r] = \infty$

we have: every limit point is a stationary point.

- Remark 1: a constant stepsize $0 < \alpha < 2/L$ is fine, and fluctuating in a range is also fine.
- Remark 2: more general descent directions are allowed (Bertsekas, Prop. 1.2.3).

We focus on $\alpha = 1/L$ in the following.

SECTION 2.2

Convergence Speed

the rate, the iteration count

THEOREM 1

The Convergence Rate of Gradient Descent

A curvature upper bound and a positive lower bound give a geometric contraction.

Theorem 1. Suppose F is L-smooth and μ-strongly convex, and let $\kappa = L/\mu$.

Then gradient descent with step $\alpha = 1/L$ satisfies, for every r,

$$\|w[r+1] - w^\star\|^2 \;\le\; \left(1 - \frac{1}{\kappa}\right)\|w[r] - w^\star\|^2$$

$$F(w[r+1]) - F^\star \;\le\; \left(1 - \frac{1}{\kappa}\right)^2 \left(F(w[r]) - F^\star\right)$$

The distance contracts by 1 − 1/κ per step; the function value by its square.

The bound controls a relative contraction — the initial error still matters.

From a Rate to an Iteration Count

For a strongly convex function, how many iterations are needed to achieve a target error?

Corollary 1. $$\frac{F(w[r]) - F^\star}{F(w[0]) - F^\star} \le \left(1 - \frac{1}{\kappa}\right)^{2r} \le \varepsilon \quad \Longrightarrow \quad r \ge \tfrac{1}{2}\,\kappa \ln \frac{1}{\varepsilon}.$$

Remark: the error we check here is relative — a smaller initial error F(w[0]) − F* means a smaller error at the same iteration.
In practice, people often check the absolute error F(w[r]) − F*.

A Two-Dimensional Diagonal Quadratic

To understand convergence, we start with a simple example: a two-dimensional quadratic function with a diagonal Hessian.

$$\min_w F(w) \triangleq \frac{1}{2} w^\top Q w, \qquad Q = \mathrm{diag}(\lambda_1, \lambda_2)$$

$$L = \lambda_1 \geq \lambda_2 = \mu > 0, \qquad \text{global minimum } w^\star = 0$$

$$w = (a; b), \qquad w[r] = (a[r]; b[r])$$

Matlab notation: (v_1, v_2) is a row vector, (v_1; v_2) is a column vector.

The Iterates, in Closed Form

For this example, the gradient descent update can be written as:

$$w[r+1] = w[r] - \alpha \nabla F(w[r]) = w[r] - \alpha Q w[r] = (I - \alpha Q) w[r] = \big((1-\alpha\lambda_1) w_1[r];\ (1-\alpha\lambda_2) w_2[r]\big)$$

from $w[r+1] = w[r] - \alpha \nabla F(w[r])$ we get

$$w[r] = (a[r]; b[r]) = \big((1-\alpha\lambda_1)^r a[0];\ (1-\alpha\lambda_2)^r b[0]\big)$$

Each coordinate is multiplied by its own factor $1 - \alpha\lambda_i$ at every step.

Corollary 2: Convergence and the Rate

Fact 1

$$\text{For } \rho \in \mathbb{R}: \quad \{\rho^r\} \text{ diverges when } |\rho| > 1, \text{ and converges when } |\rho| < 1.$$

Corollary 2

$$\|w[r]\| \text{ converges when } \rho_1 \triangleq |1 - \alpha\lambda_1| < 1 \text{ and } \rho_2 \triangleq |1 - \alpha\lambda_2| < 1.$$

$$\text{its speed of convergence depends on } \max\{\rho_1, \rho_2\}$$

- ‖w[r]‖ diverges when $\alpha > 2/\lambda_1 = 2/L$ (if $a[0] \neq 0$), and converges to 0 when $\alpha < 2/\lambda_1$.
  In theory we often pick $\alpha = 1/L$.
- When $\alpha = 1/L$, the rate is $\max\{\rho_1, \rho_2\} = \max\{0, 1 - 1/\kappa\} = 1 - 1/\kappa$.

$1 - 1/\kappa$ is almost the best we can do — it improves only to $1 - 2/(1 + \kappa)$, which changes the iteration complexity from $\tilde{O}(\kappa)$ to $\tilde{O}((1 + \kappa)/2)$; we will not worry about that factor of two.

WITH A LINEAR TERM

The Shift Does Not Matter

The linear term looks as if it should matter.

Compare the update at w[r] with the same update written at the minimizer:

$$f(w) = \tfrac{1}{2} w^\top Q w + b^\top w, \qquad \nabla f(w) = Qw + b$$

$$w[r+1] = w[r] - \alpha \nabla f(w[r]) = w[r] - \alpha\left(Qw[r] + b\right)$$

$$w^\star = w^\star - \alpha\left(Qw^\star + b\right) \qquad \text{(the update at } w^\star \text{ changes nothing)}$$

$$\text{subtracting,} \qquad w[r+1] - w^\star = (I - \alpha Q)\left(w[r] - w^\star\right)$$

Qw* + b = 0, so the two linear terms cancel exactly; the error w[r] - w* follows the same quadratic update.

The Diagonal Quadratic

For a diagonal quadratic, each coordinate follows the same one-dimensional rule.

$$x[r+1] = (I - \alpha Q)x, \qquad Q = \operatorname{diag}(\lambda_1, \ldots, \lambda_n), \quad n \geq 1$$

$$x_i[r+1] = (1 - \alpha\lambda_i)x_i, \qquad \frac{\|x[r]\|}{\|x[0]\|} \leq \max_i |1 - \alpha\lambda_i|^r$$

Coordinate i is multiplied by $1 - \alpha\lambda_i$ at each step; at $\alpha = 1/\lambda_{max}$, the slowest factor is $1 - \lambda_{min}/\lambda_{max}$.

SECTION 2.3

Why the Rate Has This Form

the two mechanisms, and the zig-zag picture

MECHANISM 1

Why a Step Above 2/L Diverges

Consider a nonzero coordinate with curvature L in a quadratic.

$$F(w) = \tfrac{1}{2} L w_1^2, \qquad \alpha L > 2$$

$$w_1[r+1] = (1 - \alpha L)\, w_1[r], \qquad |1 - \alpha L| > 1 \implies |w_1[r]| \to \infty$$

$$\alpha = \frac{2}{L}: \qquad w_1[r+1] = -w_1[r] \quad \text{(the iterates swap sides for ever)}$$

This is the largest curvature in the problem, and it is the first thing to break.

Every direction is stable when 0 < α < 2/L in this positive definite quadratic.

MECHANISM 2

Where the Condition Number Comes From

At α = 1/L, large curvature limits the step and small curvature slows progress.

the two-dimensional example: $\lambda_1 = L,\ \lambda_2 = \mu, \qquad \alpha = \frac{1}{L}, \qquad \kappa = \frac{L}{\mu}$

$$a[r+1] = \left(1 - \frac{\lambda_1}{L}\right) a[r] = 0 \ \text{(one step)}, \qquad b[r+1] = \left(1 - \frac{\lambda_2}{L}\right) b[r] = \left(1 - \frac{1}{\kappa}\right) b[r]$$

the slow coordinate sets the pace: $\approx \frac{1}{2}\,\kappa$ steps per factor e

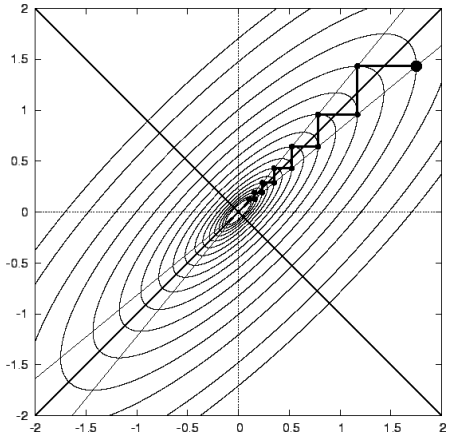
Scaling all eigenvalues equally leaves κ unchanged, once α is adjusted to 1/L.

Two rowers, one boat, one rhythm: the pace is set by whoever would capsize if you pushed harder, and the stroke count depends on the ratio of the two, not on how fast either rows.

IN PRACTICE

Zig-zag Behaviour of GD

The axes are two parameter coordinates; the ellipses are equal-loss contours.



each step perpendicular
to the last

The gradient is perpendicular to the local contour — it can point across the valley while progress along the valley stays slow.

The gradient is perpendicular to the contour, so on an elongated ellipse it points across the valley: the bouncing is the fast mode dying, the slow crawl along the floor is the small eigenvalue at 1 − 1/kappa.

SECTION 2.4

Two Paradoxes

where the bound and the measured runs disagree, and what survives without strong convexity

Paradox 1: Kappa Is Infinite, and GD Converges

What if Q = diag(λ_1, 0)?

$$Q = \mathrm{diag}(\lambda_1, 0) \qquad \Longrightarrow \qquad \kappa = \frac{\lambda_1}{0} = \infty$$

- Paradox 1: Corollary 1 implies we need an infinite number of iterations?
- GD should work for a convex problem; it shall not be infinity.
  Is the theorem too loose?

The theorem does not apply here, and we need a modification.

Resolving Paradox 1: Only Non-Zero Eigenvalues Count

If we check the function value, then GD indeed works.
For simplicity assume w[0] = (1; 1):

$$F(w) = \lambda_1 a^2 + \lambda_2 b^2 \implies F(w[r]) = \lambda_1(1-\alpha\lambda_1)^r a[0] + \lambda_2(1-\alpha\lambda_2)^r b[0] = \lambda_1(1-\alpha\lambda_1)^r + \lambda_2(1-\alpha\lambda_2)^r$$

Claim: when $Q = \mathrm{diag}(\lambda_1, 0)$, $\Phi[r] \le (1-\alpha\lambda_1)^r \Phi[0]$, $\Phi[t] \triangleq F(w[t]) - F(w^\star)$

This analysis tells us how to proceed for a general quadratic function: we should consider a modified condition number, κ_{nz}.

general convex quadratic: $\kappa_{\mathrm{nz}} \triangleq \dfrac{\lambda_{\max}}{\lambda_{\min,\mathrm{nz}}}$, $\lambda_{\min,\mathrm{nz}}$ the smallest non-zero eigenvalue

For Q = diag(λ_1, 0), κ_{nz} = 1.

Paradox 2: A Small Eigenvalue Is Worse Than None

What if Q = diag(1, 10^{-10})?
What about Q = diag(1, 10^{-20})?

$$\kappa = 10^{10} \text{ and } 10^{20} \quad \Longrightarrow \quad \text{Corollary 1 asks for } 10^{10} \ln \frac{1}{\varepsilon} \text{ and } 10^{20} \ln \frac{1}{\varepsilon} \text{ iterations}$$

- Paradox 2: as λ_2 diminishes, the iteration complexity increases to infinity; but when λ_2 reaches 0 the complexity suddenly jumps to 1 (since $\kappa_{nz} = 1$)?

Resolving Paradox 2: The Error Is a Sum of Modes

Revisit $\lambda_1 = 1$, $\lambda_2 = 10^{-10}$ with $w[0] = (1; 1)$:

Solution: tiny eigenvalues do NOT matter.

$$F(w[r]) = (1-\alpha)^r + 10^{-10}(1-10^{-10}\alpha)^r \leq (1-\alpha)^r + 10^{-10}$$

If the target error is 10^{-5}, the second term does not matter: the first term dominates, so the effective convergence rate is $1 - \alpha$.

Only when the first mode reaches $(1 - \alpha)^r = 10^{-10}$ does the second mode, with rate $1 - 10^{-10}\alpha$, become dominant.

When There Is No Strong Convexity

Without convexity, we can still bound the smallest gradient norm along a GD run.

Theorem 2. Suppose F is differentiable and L-smooth, and run GD with stepsize $1/L$.

$$\min_{0\le r\le T}\left\|\nabla F(w[r])\right\| \le \sqrt{\frac{2L\left(F(w[0])-F^\star\right)}{T}}$$

Corollary 3. With $T \ge \dfrac{2L\left(F(w[0])-F^\star\right)}{\varepsilon^2}$, $\min_{0\le r\le T}\left\|\nabla F(w[r])\right\| \le \varepsilon$

The guarantee concerns stationarity, so ε now measures gradient norm — a small gradient alone does not certify a local or global minimum.

SECTION 3

Data Preprocessing

The same data can give GD very different progress.

A Simple Example in Regression

Let's take a look at a simple example: predicting the price of a house given its saleable area.

- Use 17 Hong Kong Island transactions from March to August 2021.
- Area is measured in square feet, and price in million HKD, as the data arrives.
- For example, 954 sq ft sold for 26.50 million HKD.
- We will find the slope and intercept with gradient descent.
- All 17 transactions are listed on the next page.

THE DATA

The 17 Houses

One row is one house i: $x_i = (\text{area}_i, 1)^T$ are its inputs, y_i is its price, i = 1, …, 17.

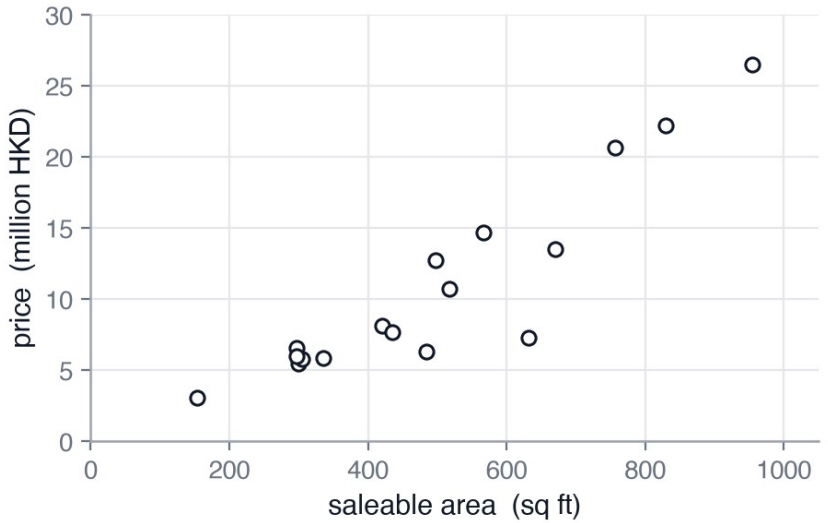
Area (ft²)	Price (HKD)	Area (ft²)	Price (HKD)
954	26.5×10^6	518	10.7×10^6
756	20.66×10^6	830	22.2×10^6
153	3.06×10^6	297	6.58×10^6
336	5.85×10^6	631	7.28×10^6
420	8.1×10^6	305	5.8×10^6
567	14.65×10^6	497	12.75×10^6
484	6.3×10^6	297	6×10^6
435	7.65×10^6	670	13.5×10^6
300	5.46×10^6		

all 17 transactions: saleable area (sq ft), price (HKD) — sorted by date, every 303rd of 5,154 Centaline Hong Kong Island sales, Mar–Aug 2021

The same 17 points as the scatter and the fit that follow.

A Simple Example in Regression

Each column $x_i = (\text{area}_i, 1)^T$ is one house; $w = (w_1, w[0])^T$.



Data matrix $X \in R^{2\times17}$: first row the area data, second row all 1.

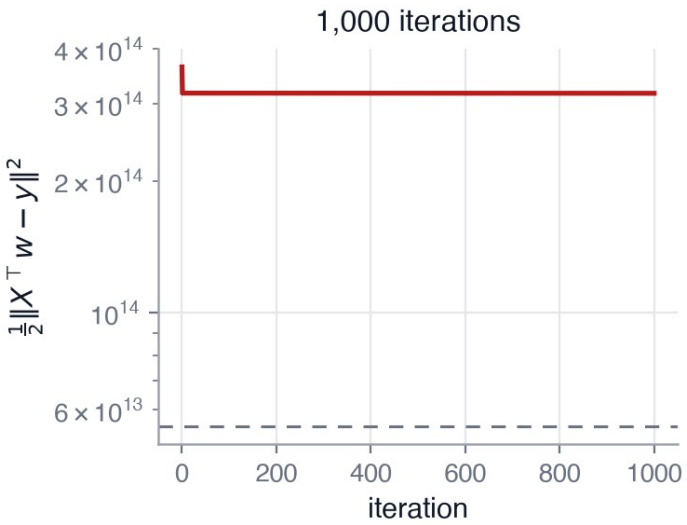
The model $\hat{y}_i = w\, x_i + b$ estimates y_i.

Problem: $\min_{w,b} \Sigma_i (w\, x_i + b - y_i)^2$, or $\min_{w,b} \|X^T w - Y\|_F^2$, where $X = [x_1 \dots x_{17}] \in R^{2\times17}$ and $Y = [y_1 \dots y_{17}] \in R^{1\times17}$.

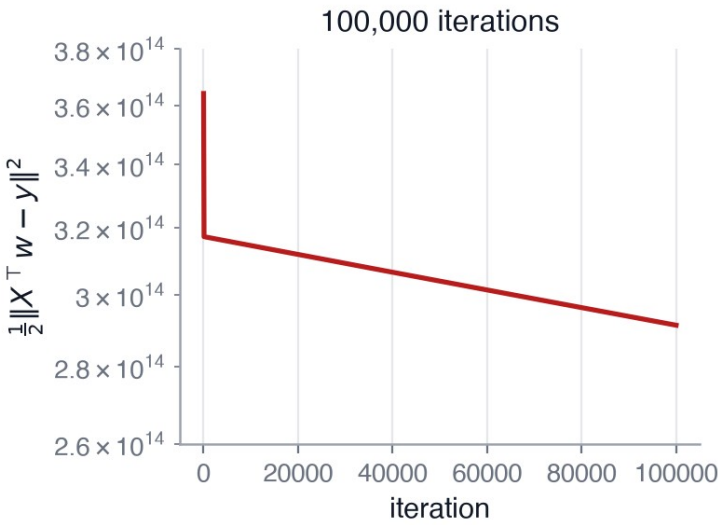
First Try

Use $\alpha = 1/\lambda_{max}(XX^T)$, starting from a constant prediction of the mean price.

Nothing is scaled:



1,000 iterations: the objective looks flat — the error seems quite large.

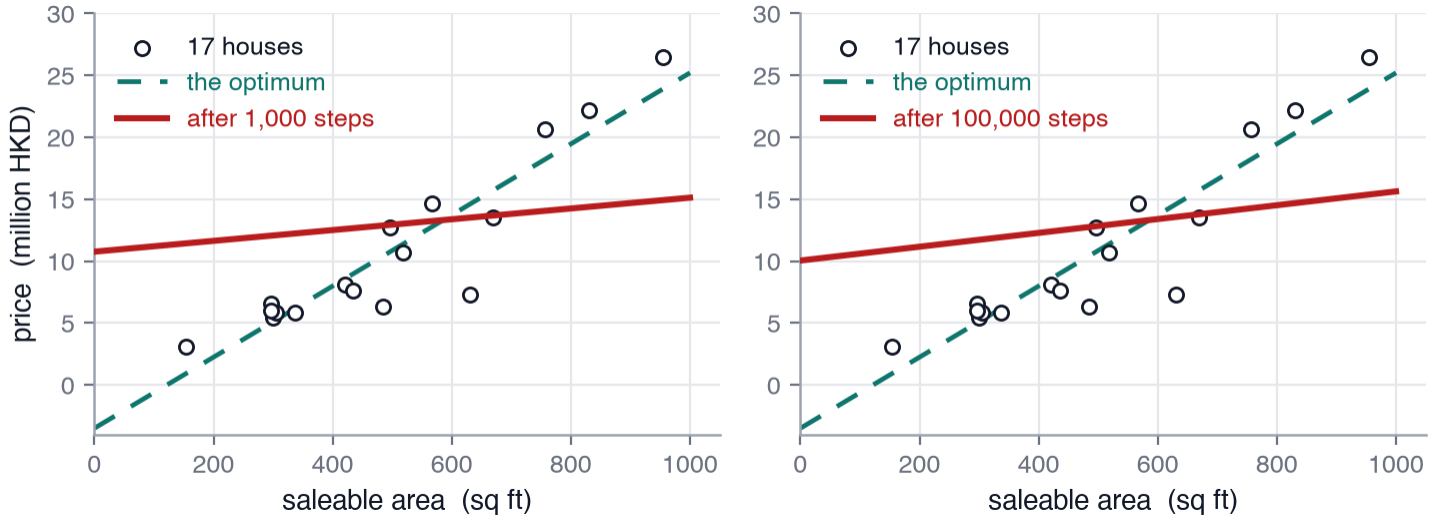


so we keep drawing: 100,000 iterations — it is working, slowly.

After 1,000 steps, the loss remains far above its minimum; after 100,000, the same run is still decreasing slowly.

First try: fitting

After 1,000 steps the fitted line is far from the least-squares optimum — and after 100,000 steps it has barely moved:



left: the fit after 1,000 steps (relative gap 0.85); right: the same run after 100,000 steps (relative gap 0.77) — it is working, but slowly

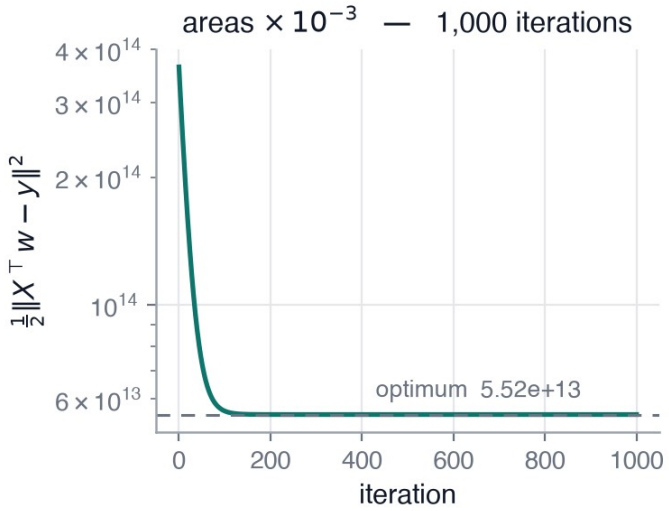
XX^T has eigenvalues 2.58 and 4.95×10^6, so $\kappa \approx 1.9 \times 10^6$.

Red is the GD fit; the dashed line is the best fit within this straight-line model.

DIAGNOSIS

Second Try

Measure area in thousands of square feet — multiply by 10^{-3}.
Same data, same initial prediction, same stepsize rule:

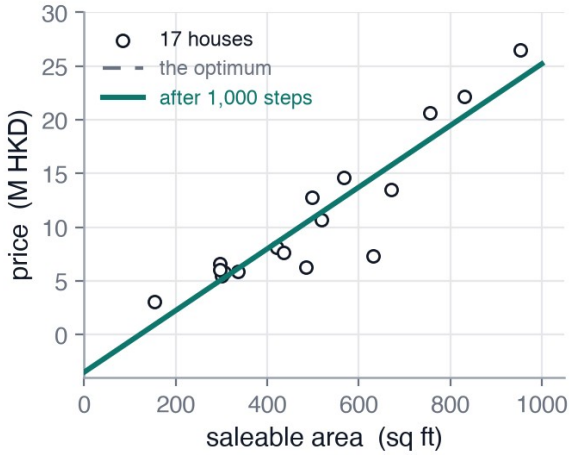


areas × 10^{-3}, 1,000 iterations

The eigenvalues of XX^T are now 0.59 and 21.4, so κ falls to about 36.
The loss seems to be stuck within the 1,000-step budget — but how do we know this is a global minimum?
In this special example we can visualize the fit; see the next page.

Second try: fitting

After 1,000 steps, the computed solution gives a fitted line that seems to fit the data pretty well.



the same 17 houses, fitted

So the solution is close to the global minimum — but note that the optimum itself is never known.

This way of reading the fit is very special: it cannot be used, at least not directly, when the input dimension is larger than 2.

Rescaling the feature does not change the set of possible fitted lines;
it changes how quickly GD reaches one.

DIAGNOSIS

A Flat Loss Curve: Has GD Converged?

Three possible explanations

- A. GD has reached the global minimum.
- B. GD has reached a suboptimal local minimum.
- C. GD has not converged yet.

Use convexity

- linear least squares: no suboptimal local minimum.
  What remains are A and C.
  The question becomes: A or C?

Two checks

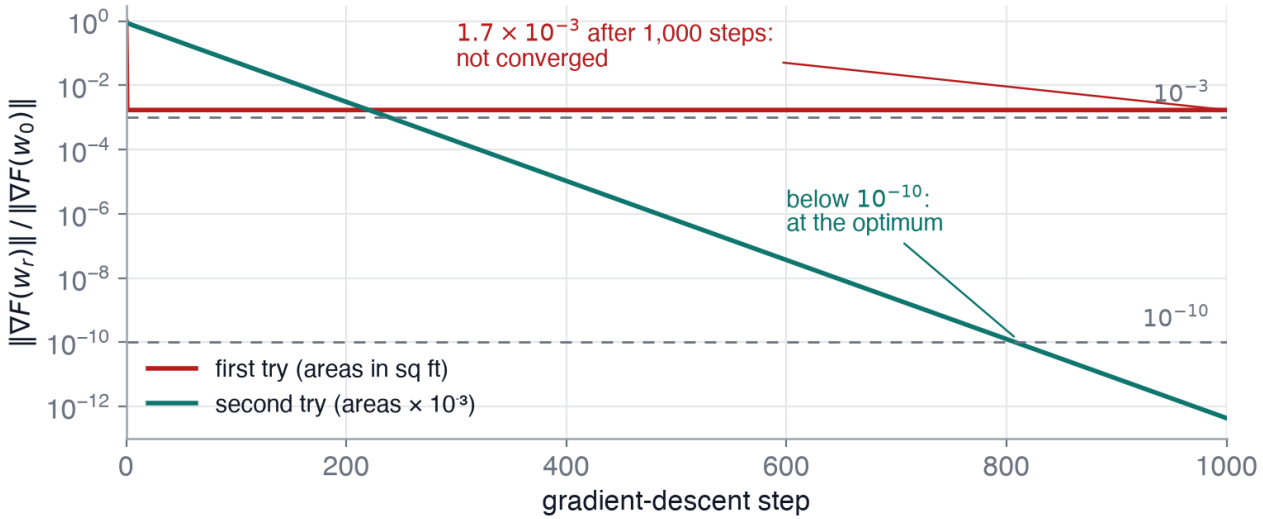
- compare the gradient norm and fitted line with a least-squares solution; a nonzero optimal residual differs from an optimization error.

Here the poorly fitted line supports C — a plateau alone would not be enough.

DIAGNOSIS

The Gradient Norm, Measured

One number separates the two runs:



relative gradient norm over 1,000 steps: the raw run settles at 1.7×10^{-3}, above the tolerance; the rescaled run crosses 10^{-10} at step 808

Same data, same start, same 1,000-step budget — only the unit of one column changed.

The gradient-norm test on the previous page, applied: above 10^{-3} not converged, below 10^{-10} done.

DIAGNOSIS

Estimating Reasonable Time for Convergence

The loss is flat and the gradient norm is still 1.7×10^{-3}.
How do we know 100× more time would bring it down — and why not a local minimum?

- In this case: convex, so no local minima exist.
  But what about other cases?

- **Tip: use the condition number to estimate —** theory predicts the step count from κ.

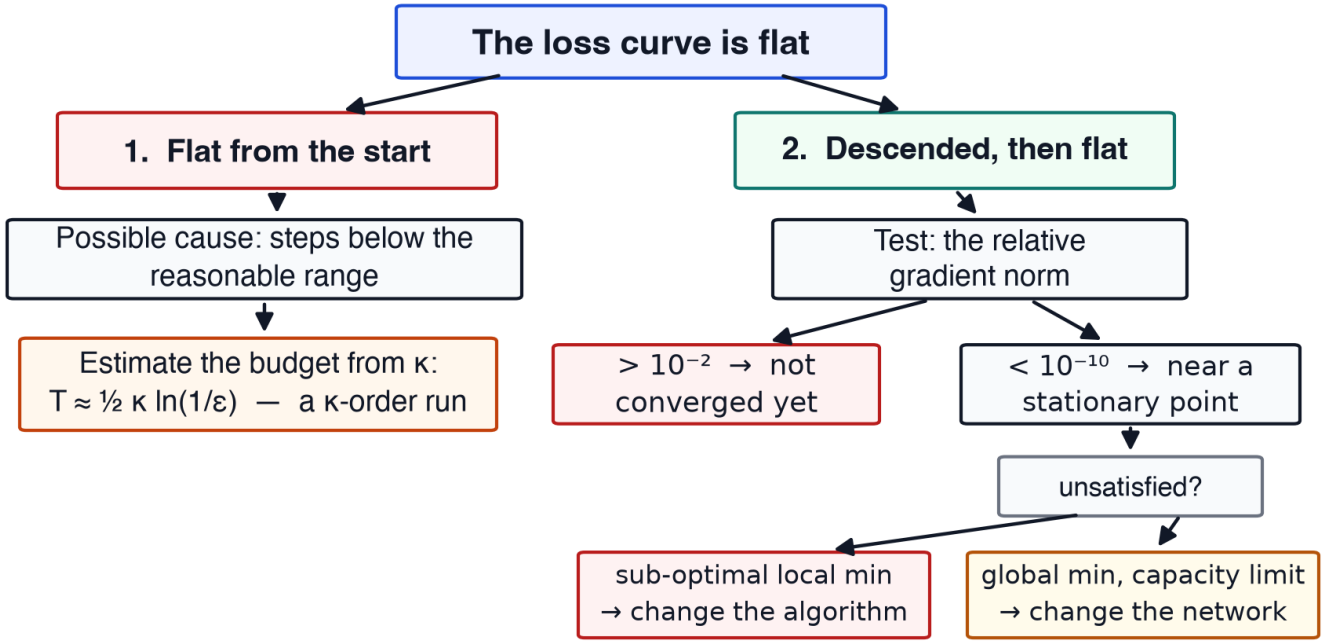
- $T \approx \frac{1}{2}\kappa \ln(1/\varepsilon)$; $\kappa(XX^T) \approx 1.9 \times 10^6$ here — so the reasonable order is 10^6 or more steps.
  100× more (10^5) will keep decreasing; convergence needs millions.

- **Disclaimer —** reality is not this simple — but it gives an idea.

DIAGNOSIS

If the Loss Is Flat: a Flowchart

Two cases, two tests, and what to change in each:



Case 1: the steps are below the κ-order range — estimate the budget.
Case 2: read the relative gradient norm.
Above 10^{-2}: not converged. Below 10^{-10}: near a stationary point.
Still unsatisfied: change the algorithm, or the network.

Scale of Loss: Commonsense in Optimization

The first run reports a loss as high as 10^{14}.
Where does such a number come from?

$$y \approx 2 \times 10^7 \text{ HKD}, \quad \hat{y} = y + 10^6 \ \ (10\% \text{ relative error}), \quad (\hat{y} - y)^2 = 10^{12}$$

- A 10% error on one house already contributes 10^{12}; seventeen houses sum to the 10^{13}–10^{14} we saw.
- The number is the unit of price, squared — nothing mysterious.
- **Lesson 1 —** in training theory, know where every number comes from.
- **Lesson 2 —** the loss can be misleading.

Scaling y to Make the Loss Smaller

Two standard remedies when the raw loss reads 10^{14}:

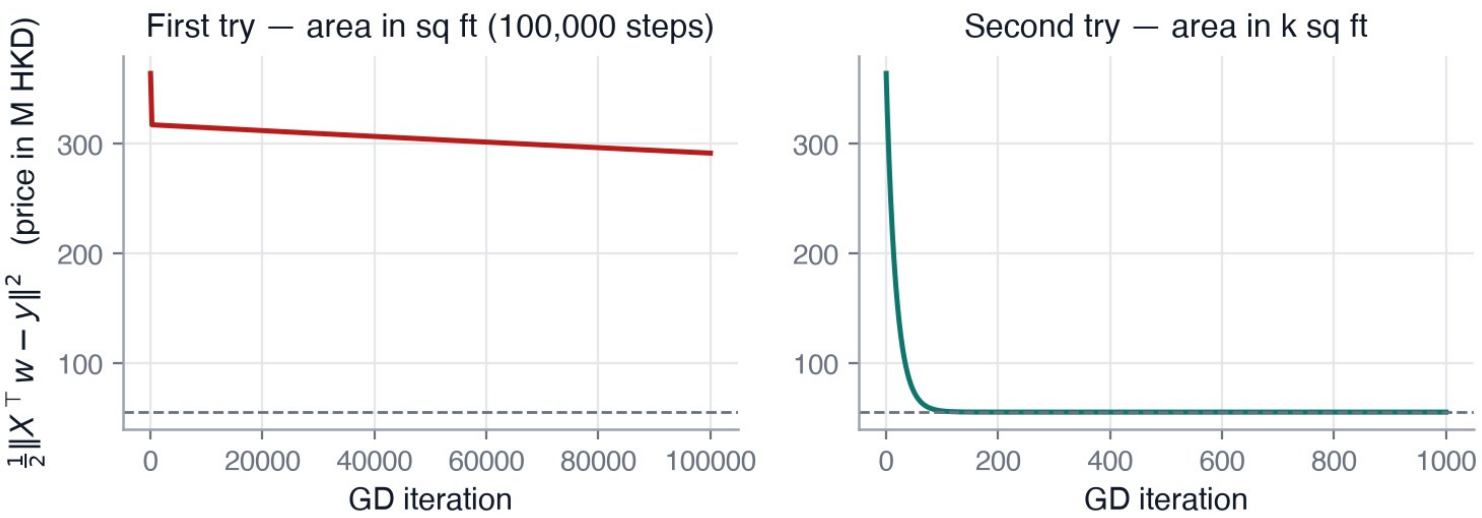
- **Plot a relative error**
  - F(w[T])/F(w[0]) — most people forget to.
  - Strictly it should be (F(w[T]) − F*)/(F(w[0]) − F*) → 0, but in practice F* is unknown.
- **Scale y**
  - change the unit — millions of HKD instead of HKD.
  - It changes the nominal values of the objective function, but does not affect the convergence of gradient descent in essence.

The same two runs, redrawn with price in millions:

SCALE OF LOSS

The Same Two Runs, Loss in Millions

Price reported in millions of HKD instead of HKD — the loss divides by 10^{12}:



left: the first try run out to 100,000 steps — 364 down to 291, still descending (relative gap 0.77); right: the second try reaches the minimum (dashed) in about 175 steps

Scaling y changes the nominal values of the objective function; it does not affect the convergence of gradient descent.

Scaling X does.

Some Training Tips

Whatever the case

- standardize the data.
- strictly, if you can, computing the condition number of XX^T is even better.

If the loss is very large — say above 10^6 — is it reasonable?

- understand the loss function and y.
- analyze the scale of y; estimate what order of loss that scale implies.

If the loss is flat, three possibilities:

- steps fewer than the reasonable range;
- steps are many, but not yet enough to reach a stationary point;
- a local optimum — or the network's representation is the limit.

Data Preprocessing: Normalization

- The condition number being large, or ill-conditioning, can arise when the variables are scaled incoherently.

- **Example —** feature 1: area, unit m^2, nominal value 120.
  - feature 2: age, unit “day”, nominal value 6000.
  - feature 3: # of bedrooms, nominal value 3.

- **“Data preprocessing” —** preprocess the data so that the data matrix has a better property (e.g. better condition number).

Normalization: Unit-Norm Rows

- Suppose $X = [x[1], x[2], \ldots, x[n]] \in R^{d \times n}$.
- Each column represents a sample.
- Each row represents a feature.
- **Normalization —** scale each row so that each row has unit norm.
- **Lesson —** normalizing data is reducing condition number, which makes GD faster.

Normalization and Preconditioning

What is the effect of normalization?
Assume $X^T = [X_1, \ldots, X_d]$.

- Linear regression $\|X_1 w_1 + \ldots + X_d w_d - b\|^2$.
- Convergence speed depends on $\kappa(XX^T)$.
- Scale each row X_i^T by $\gamma_i = 1/\|X_i\|$, $i = 1, \ldots, d$.
- Equivalent to: change matrix X to $\hat{X} = \Gamma X$.
- For this $\hat{X}\hat{X}^T = \Gamma X X^T \Gamma^T$, all of its diagonal entries are 1.
- Convergence speed now depends on $\kappa(\hat{X}\hat{X}^T)$.

Jacobi Preconditioning

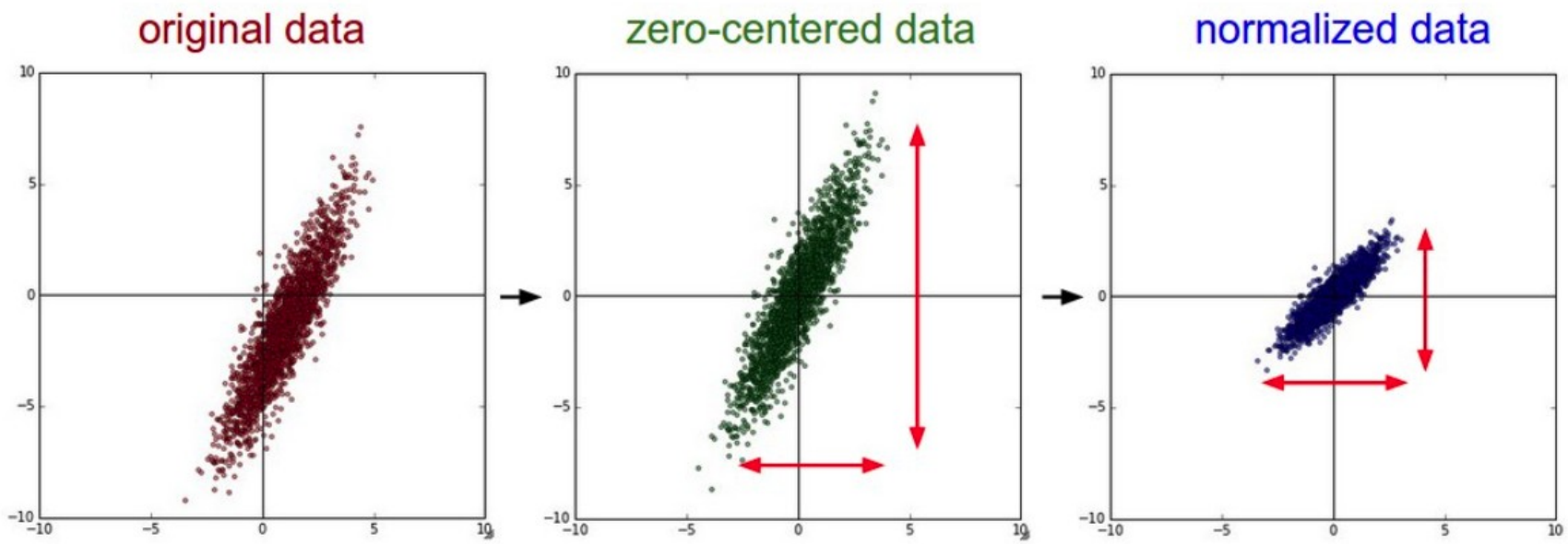
- e.g. $X = [100, 0; 0, 1]$, what is $\hat{X}$?
- Definition: the Jacobi preconditioner of a matrix is its diagonal.
  Here the matrix is XX^T, so $\Gamma = \mathrm{diag}(XX^T)^{-1/2}$.
- Called Jacobi preconditioning in numerical linear algebra.
- **Does it improve condition number always? —** A counter-example (e.g. an example that shows Jacobi preconditioning makes condition number worse) can be easily found by current AI models.
  Also mentioned in R. Sun, Y. Ye, Worst case complexity of Cyclic Coordinate Descent: $O(n^2)$ Gap with Randomized Version. MP 2019.

Centering

Example: predict which university to get in.

- **feature 1: score —** X_1^T = [630, 650, 640, 660].
- After normalization: (0.488, 0.503, 0.496, 0.511); quite close.
- **Centering —** subtract each entry by the mean of all data.
- **E.g. —** X_1^T = [630, 650, 640, 660].
- After centering: −15, −5, 5, 15.
- Further after normalization: −0.67, −0.22, 0.22, 0.67.
- **Claim (informal) —** centering can also reduce condition number.
- **Exercise —** compare matrix with entries in Unif[0, 1] and matrix with entries in Unif[−0.5, 0.5].

Data Preprocessing: Figure Illustration



Common data preprocessing pipeline. Left: Original toy, 2-dimensional input data. Middle: The data is zero-centered by subtracting the mean in each dimension. The data cloud is now centered around the origin. Right: Each dimension is additionally scaled by its standard deviation. The red lines indicate the extent of the data - they are of unequal length in the middle, but of equal length on the right.

Figure: Data preprocessing pipeline. Source: Stanford CS231n course notes.

Standardization For Linear Regression

For the linear regression problem, where X = (x[1], …, x[n]) is d × n.
Define an operator η: $R^n \to R^n$ with i-th element

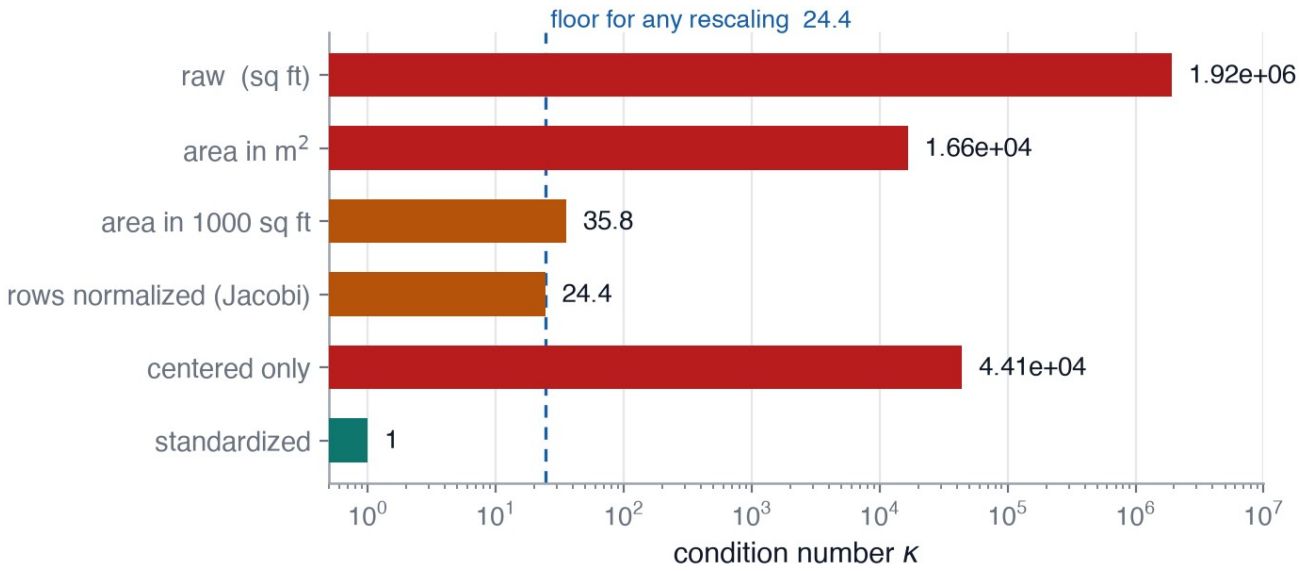
$$\min_w \sum_{i=1}^{n} \|y - X^\top w\|^2$$

$$\eta(h)_i \triangleq \frac{h_i - \mu}{\sigma}, \qquad \mu = \frac{1}{n}\sum_i h_i, \qquad \sigma = \frac{1}{n}\sum_i \|h_i - \mu\|^2$$

- Then define a new matrix $\tilde{X} = [\eta(X_{1,:}); \ldots; \eta(X_{d,:})]$ by scaling each column of X (i.e., each row of X^T).
- Solve the new problem $\min_{\tilde{w}} \Sigma_i \|y - \tilde{X}^T\tilde{w}\|^2$.
- **Inference —** how to generate a prediction y_{test} for a new data point x_{test}?
- Use μ, σ for each feature, to process each entry of x_{test}, to get $\tilde{x}_{test}$; then get prediction $y_{test} = \tilde{x}_{test}^T\tilde{w}$.

Different Ways Lead to Different Condition Numbers

A different scaling factor and whether using centering or not will give you a different condition number.



kappa from 1.9×10^6 down to 1 - and the floor at 24.4

In this two-feature design, rescaling alone stops at $\kappa \approx 24.4$; centering and standardizing the area feature can reach $\kappa = 1$.

Why Centering Gives κ = 1

$$y_i = w x_i + b \implies y = X^\top \theta, \qquad X = \begin{bmatrix} x^\top \\ \mathbf{1}^\top \end{bmatrix} \in \mathbb{R}^{2\times n}, \quad \theta = (w, b)$$

$$XX^\top = \begin{bmatrix} x^\top x & x^\top \mathbf{1} \\ \mathbf{1}^\top x & \mathbf{1}^\top \mathbf{1} \end{bmatrix} = \begin{bmatrix} \sum_i x_i^2 & \sum_i x_i \\ \sum_i x_i & n \end{bmatrix}$$

$$\text{centering } \tilde{x} = x - \bar{x}\mathbf{1} \implies \tilde{x}^\top \mathbf{1} = 0 \implies XX^\top = \begin{bmatrix} \|\tilde{x}\|^2 & 0 \\ 0 & n \end{bmatrix}$$

$$\text{scaling } \sigma^2 = \tfrac{1}{n}\|\tilde{x}\|^2 \implies \|\tilde{x}/\sigma\|^2 = n \implies XX^\top = nI_2 \implies \kappa(XX^\top) = 1$$

- The model has two rows: the feature x and the intercept 1.
- Centering kills the off-diagonal term $x^\top 1$; scaling makes the two diagonal terms equal. Both are needed, and the off-diagonal is the one that matters here.
- Without centering, row normalization leaves $\kappa = (1 + \rho)/(1 - \rho)$ with $\rho = x^\top 1/(\|x\|\sqrt{n}) = 0.921$ for these 17 houses — that is the floor 24.4 on the previous page.

A SECOND LAB

Condition Number vs Standardization Lab

- ruoyus.github.io/me2605/demos/condscale-lab/
- In the experiment you generate a random d × n matrix X, form XX^T, and compute $\kappa_{nz} = \lambda_{max}/\lambda_{min,nz}$.
- You choose n, d, the seed, and the distribution — Uniform[1, 100], Gaussian, Cauchy.
- **Change the preprocessing —** you can change how much to scale, and how much to subtract, in the website.
- **Check how κ changes —** as the input changes.

SECTION 4

Explaining Normalization

NOTATION

Review: the MLP Network Expression

MLP (multi-layer perceptron), also called a fully connected network (FCN).
This is the network architecture we have used so far:

$$z_0 = x,$$

$$h_l = W_l z_{l-1} + b_l, \quad l = 1, \ldots, L \qquad \text{(pre-activation)}$$

$$z_l = \phi(h_l), \quad l = 1, \ldots, L-1 \qquad \text{(post-activation)}$$

$$f_\theta(x) = h_L, \qquad \theta = \{W_l, b_l\}_{l=1}^{L}.$$

$x \in \mathbb{R}^{d_0}$; W_l: $d_{l-1} \to d_l$; b_l, h_l, $z_l \in \mathbb{R}^{d_l}$.

L includes the output layer: there are $L - 1$ hidden layers.

The Same Loss, Grouped Two Ways

How to extend this idea to deep neural-nets?

Consider the loss for a network with one hidden layer: $F(\theta) = \|Y - W_2\varphi(W_1X)\|^2_F$.

- As usual, we will need to standardize each row of X, to speed up the convergence of W_2, W_1.
- Besides, the loss can be written as $F(\theta) = \|Y - W_2Z_1\|^2_F$,
  - where $Z_1 \in \mathbb{R}^{d_1 \times n}$ is the collection of the post-activations $z_1[1], \ldots, z_1[n]$ for each input data $x[1], \ldots, x[n]$.

THE KEY SENTENCE

Convergence Speed and the Hessian for W_2

Intuitively:

the convergence speed of W_2 shall depend on the condition number of $(Z_1)^\top Z_1$, thus it is reasonable to standardize each row of Z_1.

For each output row of W_2, the Hessian is $2Z_1Z_1^\top$; its positive spectrum controls GD convergence in the directions that affect the fit.

Two Hidden Layers

Now consider the loss for a network with two hidden layers:

$$F(\theta) = \left\| Y - W_3 \, \varphi(W_2 \, \varphi(W_1 X)) \right\|_F^2$$

- Besides standardizing each row of X, we can write $F(\theta) = \|Y - W_3 Z_2\|_F^2$ — then it is reasonable to standardize each row of Z_2.
- We can also write it as $F(\theta) = \|Y - W_3 \varphi(W_2 Z_1)\|_F^2$ — thus hoping to standardize each row of Z_1.

THE GOAL

Standardize Every Row of Every Layer

For the L-layer loss:

$$F(\theta) = \left\| Y - W_3\, \varphi(W_2\, \varphi(W_1 X)) \right\|_F^2$$

- **Goal —** we hope to standardize each row of Z_l, l = 0, 1, 2, …, L.
- j-th row of Z_l is the collection of the output of the j-th neuron after the l-th layer.
- Z_0 = X is the input.

THE SETUP

Computing the Condition Number of a One-Hidden-Layer Network

One fixed post-activation matrix; three versions of it; one number each:

$$Z = \phi(W_1 X) \in \mathbb{R}^{32\times 256}$$

- **The network —** a network with one hidden layer, $f(x) = W_2\varphi(W_1x)$; the last layer fits $Y \approx W_2Z$, so each row of W_2 sees the Hessian $2ZZ^T$.
- **Z —** the post-activation matrix of the hidden layer — one column per sample. 32 outputs, 256 samples, one fixed draw (seed 7).
- **The candidates —** post-activation BN standardizes each row of Z (per feature, across samples); post-activation LN standardizes each column (per sample) — call the results Z_{BN} and Z_{LN}.
- **The number —** κ_{nz} of $Z_{BN}\,Z_{BN}^T$ and $Z_{LN}\,Z_{LN}^T$ — the condition number over the non-zero eigenvalues.

Every number on the next two pages comes from this one matrix.

MEASURED

Case One: the Outputs on Many Scales

32 outputs, 256 samples, and scales over six orders of magnitude:

	κ_{nz}	exact zeros	row-energy ratio
nothing	3.9×10^7	12	1.9×10^{12}
BatchNorm	3.7	0	1.00
LayerNorm	4.4×10^7	8	151

The same 32 × 256 ReLU features, with row scales from 10^{-3} to 10^{3}.
Full-batch BN equalizes row energies, but correlations still affect κ_{nz}; LN changes the row energies, yet its positive spectrum is still poorly conditioned here.

BatchNorm is 10^7 times better here; LayerNorm cannot see it.

Disclaimer — this computation was generated by DeepSeek-V4.1-Flash; the instructor has not checked whether it is correct.

MEASURED

Case Two: the Batch Is Small

The same matrix, with BatchNorm's two numbers from B samples:

	BatchNorm: κ_{nz}	its row-energy ratio	LayerNorm: κ_{nz}
$B = 2$	11,431	5.0×10^3	3.5
$B = 4$	1,334	668	3.5
$B = 8$	111.9	66.8	3.5
$B = 32$	8.6	4.6	3.5
$B = 256$	3.7	1.00	3.5

B samples calibrate BN statistics on well-scaled features, then all 256 are evaluated.
LN stays at 3.53 because its statistics do not depend on B; entries are medians over 200 calibration draws.

LayerNorm does not move: its two numbers are exact.

Disclaimer — this computation was generated by DeepSeek-V4.1-Flash; the instructor has not checked whether it is correct.

THE LAB

Normalization Lab

We consider neural network training, and the performance of normalization layers.

- **What it is —** a ReLU MLP on a small two-class problem, plain SGD, in your browser.
- **What you choose —** the number of hidden layers (2, 4, 8, 12), the batch size, the input scale, and the method: nothing, LayerNorm, BatchNorm, RMSNorm.
  - For BatchNorm, also how it is evaluated: running statistics, or the test batch.
- **Three predefined settings —** baseline, depth with a scaled input, small mini-batches.
- **What it shows —** the test loss and accuracy curves, and the activation RMS at each depth over all 384 test samples.

- ruoyus.github.io/me2605/demos/normalization-lab/

Predict before you run it: which method performs best, and which worst?

SUMMARY

Summary

Other normalization layers, the rate, and how to choose

SUMMARY

What to Take Away

Part A — other normalization layers

- Two types of normalization layers: layer-output-based normalization and weight-based normalization.
- Layer-output-based: BN per feature, LN per sample, RMSNorm drops the mean.
- Weight-based: WN per row of W, SN by the largest singular value.

Part B — the convergence rate

- $\kappa = L/\mu$; steps $\approx \frac{1}{2}\kappa \ln(1/\varepsilon)$; any constant $\alpha < 2/L$ converges.
- a zero eigenvalue makes no contribution, so it can be ignored — hence κ_{nz}.
- a smaller eigenvalue does not contribute if it is much lower than the desired error.

Part C — analyzing a regression example

- Standard procedure: data preprocessing.
- Mechanism: changes the condition number.

Part D — relating normalization and condition number

- Data preprocessing applies to the data X; a normalization layer applies to each layer's output.
- So it is preprocessing extended to every layer — effect: smaller condition numbers.

SUMMARY

Check Yourself

- State what weight normalization and spectral normalization rescale.
- Given L and μ, predict the steps GD needs to reach ε.
- Define L-smoothness and μ-strong convexity, and write $\kappa = L/\mu$.
- Describe the three preprocessing operations, and why each one helps.
- Derive the normalization layer from the perspective of reducing κ.

Appendix

Computing the two extremes, and the two convergence proofs

APPENDIX

Eigenvalues by the Characteristic Equation

For a quadratic, the convergence speed depends on $\kappa = \lambda_{max}/\lambda_{min}$.

Here is how the two numbers are computed.

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \qquad \text{the rate depends on these two numbers}$$

$$\textbf{Characteristic equation:} \quad \det(\lambda I - A) = 0 \qquad \text{every root is an eigenvalue}$$

- **2 × 2: one quadratic —** $\lambda^2 - \mathrm{tr}(A)\cdot\lambda + \det(A) = 0$, so both roots come out exact.
  - $A = [4\ 1;\ 1\ 2]$: $\lambda^2 - 6\lambda + 7 = 0$, so $\lambda = 3 \pm \sqrt{2}$ and $\kappa \approx 2.78$.
  - XX^T, the 17-houses matrix, is 2 × 2 — that is why κ was quoted exactly there.

Remark — for a non-quadratic function κ is the largest eigenvalue over all the matrices divided by the smallest.
In practice it is estimated; not discussed here.

The Two Extremes by Iteration

To compute κ you only need the two ends:

Definition. $x_0 \neq 0, \qquad x_{r+1} = Hx_r/\|Hx_r\|, \quad r = 0, 1, 2, \ldots$

Assumptions. $Hv_1 = \lambda_1 v_1, \quad \lambda_1 > \lambda_2 \geq \cdots \geq \lambda_n, \quad \langle x_0, v_1 \rangle > 0$

Claim. $\|x_r - v_1\| = \mathcal{O}\big((\lambda_2/\lambda_1)^r\big)$

Corollary 1. $x_r \to v_1$

Corollary 2. $x_r^\top H x_r \to \lambda_{\max}(H)$

For the smallest, shift the spectrum and run the same iteration:

Shift: $k \geq \lambda_{\max}(H) \implies \lambda_{\max}(kI - H) = k - \lambda_{\min}(H), \quad$ then subtract k

One matrix-vector product per iteration; the Rayleigh quotient adds a dot product.

Result and Proof: The Linear Rate (Strongly Convex)

RESULT — LINEAR RATE

Let F be L-smooth and μ-strongly convex, and run gradient descent with any stepsize η ≤ 2/(μ+L). Then for every r,

$$\|w_{r+1} - w^\star\|^2 \;\le\; \big(1 - 2\eta\,\mu L/(\mu + L)\big)\,\|w_r - w^\star\|^2$$

Proof.

Write $e_r = w_r - w^*$; subtracting the optimum from the update leaves the error following

$$e_{r+1} = e_r - \eta\,\big(\nabla F(w_r) - \nabla F(w^\star)\big)$$

Smoothness and strong convexity, used together, give for all u and v

$$\langle \nabla F(u) - \nabla F(v),\, u - v\rangle \;\ge\; \big(\mu L\|u - v\|^2 + \|\nabla F(u) - \nabla F(v)\|^2\big)/(\mu + L)$$

Set $u = w_r$ and $v = w^*$, expand the square of the error, and substitute:

$$\|e_{r+1}\|^2 \;\le\; \big(1 - 2\eta\,\mu L/(\mu + L)\big)\,\|e_r\|^2 + \eta\,\big(\eta - 2/(\mu + L)\big)\,\|\nabla F(w_r) - \nabla F(w^\star)\|^2$$

The stepsize condition makes the last coefficient non-positive, so that term drops; at η = 1/L what is left is the factor 1 − 2/(1+κ).

Result and Proof: The Sublinear Rate (Non-Convex)

RESULT — SUBLINEAR RATE

Let F be L-smooth and bounded below by F^*, and run gradient descent with stepsize 1/L. Then for every T,

$$\min_{0 \le r \le T} \|\nabla F(w_r)\| \;\le\; \sqrt{2L\,(F(w_0) - F^\star)/T}$$

Proof.

L-smoothness with this stepsize is the descent lemma:

$$F(w_{r+1}) \;\le\; F(w_r) - \|\nabla F(w_r)\|^2/(2L)$$

Sum it over r = 0, …, T − 1; the losses telescope:

$$\sum_{r=0}^{T-1} \|\nabla F(w_r)\|^2/(2L) \;\le\; F(w_0) - F(w_T) \;\le\; F(w_0) - F^\star$$

The smallest of the T gradients is at most their average, so the sum above bounds it too.

Square roots give the stated bound; with $T \ge 2L(F(w_0) - F^*)/\varepsilon^2$ the right side is below ε, and no convexity was used anywhere.